

Smart Turntable Assistant

Daniel Kiss

2026. április 16.

Tartalomjegyzék

1	Bevezetés	3
1.1	Problémafelvetés	3
1.2	A szakdolgozat célkitűzései	4
2	Technológiai háttér és szakirodalmi áttekintés	6
2.1	Audio Fingerprinting technológiák	6
2.2	Alkalmazott keretrendszerek	7
2.3	Valós idejű kommunikáció	7
2.4	Fejlesztési és futtatási környezet	8
3	Követelményanalízis	9
3.1	Funkcionális követelmények	9
3.2	Nem-funkcionális követelmények	10
3.3	Használati esetek	11
4	Rendszerterv és architektúra	13
4.1	Kliens-Szerver architektúra	13
4.2	Adatbázis-tervezés	14
4.3	Konkurencia és szálkezelés	15
5	Megvalósítási részletek	18
5.1	Digitális jelfeldolgozás (Digital Signal Processing)	18
5.2	A vízesés elvű metaadat-stratégia	19
5.3	Backend állapotkezelés és adat-streamelés	19
5.4	Frontend interfész és reaktív architektúra	20
5.4.1	Kliens állapot és Útválasztás	20
5.4.2	Dinamikus stílusok és Témák	20
5.4.3	Szinkronizált dalszöveg-renderelés	21
5.5	Harmadik féltől származó integrációk	21

6	Tesztelés és Értékelés	22
6.1	Manuális és integrációs tesztelés	22
6.2	Teljesítmény-optimalizáció	23
7	Összegzés és jövőbeli fejlesztések	24
7.1	Az elért eredmények értékelése	24
7.2	Jövőbeli fejlesztési lehetőségek	24

1. fejezet

Bevezetés

Az elmúlt években a zeneipar az analóg fizikai hordozók, különösen a bakelitlemezek népszerűségének példátlan visszatérését tapasztalta. Az audiofileket és a hétköznapi hallgatókat egyaránt vonzza a mechanikus élmény, a szándékos cselekvés, amellyel egy albumot az elejétől a végéig meghallgatnak, valamint az analóg hangzás egyedi, meleg akusztikai jellemzője. Ugyanakkor, egy olyan korszakban, amelyet a digitális streaming platformok, mint a Spotify és az Apple Music uralnak, a hagyományos hanglemez hallgatási élmény alapvetően elszigetelt marad a modern digitális ökoszisztémától.

Ez a szakdolgozat egy Okos Lemezjátszó Asszisztens tervezését, architektúráját és megvalósítását mutatja be, amely egy szoftvervezérelt hidat képez az analóg audio hardverek és a modern digitális kényelmi funkciók között. Valós idejű jelfeldolgozás és külső API integrációk felhasználásával ez a projekt arra törekszik, hogy modernizálja az analóg zenehallgatás élményét anélkül, hogy a bakelit lemez hallgatás élményébe, hangi világába beleszólna.

1.1 Problémafelvetés

Az analóg hangzás alapvető korlátja, hogy egy folyamatos, nyers elektromos jel, amely teljes mértékben nélkülözi a metaadatokat. A digitális hangfájlokkal ellentétben egy lemezjátszó nem képes natív módon tájékoztatni a hallgatót vagy a csatlakoztatott eszközöket arról, hogy éppen mi szól. Ez a technológiai szakadék számos jól elkülöníthető problémát eredményez a modern hanglemez-hallgatók számára:

1. **A digitális integráció hiánya:** A felhasználók nem tudják automatikusan nyomon követni zenehallgatási szokásaikat. A lejátszások naplózása az olyan közösségi zenei platformokra, mint a Last.fm, fáradságos, manuális adatbevitelt igényel.

2. **A kontextuális média hiánya:** A modern streaming kényelmi funkciói, mint például a szinkronizált dalszövegek és a nagy felbontású albumborítók megjelenítése, analóg lejátszás közben teljesen elérhetetlenek.
3. **Hardveres degradáció nyomon követhetetlensége:** A lemezjátszó tűk élettartama korlátozott, amelynek lejártá után véglegesen károsíthatják a bakelitlemezek barázdáit. A felhasználók jelenleg csak durva becslésekre támaszkodhatnak a hangszedő kopásának nyomon követésében.
4. **Hozzáférhető diagnosztika hiánya:** Az olyan problémák azonosítása, mint a túlzott motorrezgés, a felületi sérülések vagy az inner-groove distortion, hagyományosan drága, dedikált akusztikai mérőműszereket igényelnek.

1.2 A szakdolgozat célkitűzései

A szakdolgozat elsődleges célja egy robusztus, valós idejű Kliens-Szerver alkalmazás megtervezése és megvalósítása, amely passzívan figyeli egy szabványos lemezjátszó audio kimenetét, és „okos” funkciók átfogó készletét nyújtja. A rendszert úgy terveztem, hogy egy USB audio interfészhez csatlakoztatott, alacsony erőforrás-igényű hardveren fusson.

A fent vázolt problémák megoldása érdekében a projekt a következő műszaki célkitűzéseket határozza meg:

- **Valós idejű jelfeldolgozás:** Egy konkurens Python backend fejlesztése, amely képes az analóg audio stream folyamatos elemzésére egy zeneszám kezdetének vagy végének detektálásához, valamint a hardverdiagnosztikai adatok matematikai kiszámításához.
- **Audio Fingerprinting és metaadat-bővítés:** Egy robusztus felismerési pipeline implementálása, amely rövid hangmintákat rögzít, és audio fingerprinting szolgáltatások segítségével azonosítja azokat. Továbbá egy waterfall elvű metaadat-lekérdező rendszer tervezése, amely megbízhatóan lekéri a pontos sáv hosszakat, albumneveket és nagy felbontású borítóképeket.
- **Automatizált digitális szinkronizáció:** Háttér szolgáltatások létrehozása az időbélyeggel ellátott dalszövegek letöltésére, valamint egy automatizált Last.fm scrobbling funkció implementálása, amely pontosan tiszteletben tartja a sáv eltelt lejátszási idejét.

- **Adatbázis- és hardverkezelés:** Egy relációs adatbázis megvalósítása a hallgatási előzmények tárolására, a testreszabott vizuális preferenciák mentésére, valamint a felhasználó által definiált lemezjátszó-hangszedők életciklusának pontos nyomon követésére.
- **Modern, reaktív felhasználói felület:** Egy reszponzív Single Page Application (SPA) tervezése Vue.js használatával, amely WebSockets technológián keresztül kommunikál a backenddel. Az interfésznek zero-latency vizuális visszajelzést kell nyújtania a felhasználók számára, és dinamikus tematikus testreszabhatóságot kell kínálnia.

2. fejezet

Technológiai háttér és szakirodalmi áttekintés

Egy valós idejű hardvermonitorozó és zenefelismerő rendszer fejlesztése számos tudományág integrációját igényli, a digitális jelfeldolgozástól a modern webalkalmazás-architektúrákig. Ez a fejezet áttekinti azokat az alapvető technológiákat, algoritmusokat és keretrendszereket, amelyeket a projekt megvalósításához választottam.

2.1 Audio Fingerprinting technológiák

Az audio fingerprinting egy olyan mechanizmus, amelyet egy hangminta azonosítására használnak egyedi akusztikai jellemzők kinyerésével és azok egy ismert adatbázissal történő összehasonlításával. Az alapvető metaadat-címkézéssel ellentétben a fingerprinting a tényleges audiotartalmat elemzi. A folyamat jellemzően az audiojel spektrogrammá alakítását, a nagy energiájú pontok vagy csúcsok azonosítását, és egy constellation map létrehozását foglalja magában. Ezeket a csúcsokat ezután kompakt digitális aláírásokká alakítják.

Ez a matematikai megközelítés rendkívül ellenálló a háttérzajjal, a hangmagasság-torzulással és a bakelitlemezekre jellemző felületi sercegéssel szemben. Ebben a projektben iparági szabványnak számító felismerő motorokat – köztük a Shazam API-t és az ACRCLOUD-ot – alkalmazok a fingerprint egyeztetés elvégzésére. Ezek a szolgáltatások a rögzített audio buffereket hatalmas kereskedelmi adatbázisokkal vetik össze, hogy az analóg tökéletlenségek ellenére is pontosan azonosítsák a lejátszott sávot.

2.2 Alkalmazott keretrendszerek

Az alkalmazás architektúrája a felelősségi körök szigorú szétválasztásán alapul, egy robusztus backend-re és egy dinamikus frontend-re osztva.

A backend esetében a választás a Pythonra esett, a matematikai számításokhoz és a digitális jelfeldolgozáshoz rendelkezésre álló kiterjedt ökoszisztémája miatt. Az olyan könyvtárak, mint a NumPy és a SciPy, kulcsfontosságúak a tömbműveletek elvégzésében, a frekvenciaszűrésben és az akusztikai metrikák valós idejű kiszámításában. A webservert réteget a Flask hajtja meg. Lightweight mikro-keretrendszerként a Flask biztosítja a szükséges eszközöket egy RESTful API kiajánlásához és az adatbázis-munkamenetek kezeléséhez a SQLAlchemy ORM-en keresztül, felesleges overhead bevezetése nélkül.

A frontend egy Vue.js alapú Single Page Application-ként került implementálásra. A Vue a reaktív, komponens-alapú architektúrája miatt bizonyult ideális választásnak, amely lehetővé teszi, hogy a felhasználói felület azonnal frissüljön a mögöttes adatok változásakor. A globális állapotkezelést a Pinia végzi, biztosítva, hogy a hardverbeállítások, a felhasználói hitelesítési állapotok és az aktuális lejátszási metaadatok tökéletesen szinkronizálva maradjanak az alkalmazás összes nézetében.

2.3 Valós idejű kommunikáció

A rendszer egyik alapvető követelménye, hogy a hardverdiagnosztikát és a szinkronizált dalszövegeket érzékelhető késleltetés nélkül jelenítse meg. A hagyományos HTTP polling, ahol a kliens ismétlődően kér adatokat a szervertől, jelentős hálózati többletterhelést okoz, és teljesen alkalmatlan a nagyfrekvenciás audio vizualizációra.

Ennek megoldására az alkalmazás WebSockets technológiát alkalmaz. A WebSockets folyamatos, full-duplex kommunikációs csatornát hoz létre egyetlen TCP kapcsolaton keresztül. Ez lehetővé teszi a Python backend számára, hogy folyamatosan adatokat toljon kifelé anélkül, hogy megvárná a kliens kérését. A Socket.IO könyvtár hidat képez a Flask backend és a Vue frontend között, elősegítve a hangerőszintek, a click detection események és a lejátszási időzítők zökkenőmentes sugárzását, másodpercenként tíz képkockás sebességgel.

2.4 Fejlesztési és futtatási környezet

Annak biztosítása, hogy a szoftver megbízhatóan fusson a különböző operációs rendszereken, szigorú környezeti izolációt igényel. A konténerizációhoz a Docker került felhasználásra, amely a teljes alkalmazást és a rendszerszintű függőségeit szabványosított egységekbe csomagolja.

Az audio alkalmazások konténerizálásának jelentős kihívása, hogy az izolált környezet számára hozzáférést kell biztosítani a host hardveréhez. Ezt a problémát a gazdagép hangeszközeinek a Docker konténerbe történő leképezésével, valamint az Advanced Linux Sound Architecture (ALSA) és a PulseAudio alrendszerek áthidalásával oldottam meg. Ez a beállítás lehetővé teszi, hogy az alkalmazás fejlesztés közben zökkenőmentesen telepíthető legyen szabványos asztali számítógépekre, miközben tökéletesen optimalizált marad a végső production environment, például egy alacsony fogyasztású, egykártyás számítógépre, mint a Raspberry Pi.

3. fejezet

Követelményanalízis

Az architekturális tervezés és a szoftverimplementációs fázisok előtt a rendszerkövetelmények átfogó elemzése szükséges. Ez biztosítja, hogy a végtermék sikeresen áthidalja az analóg hanglemez-lejátszás és a modern digitális kényelmi funkciók közötti szakadékot. Ez a fejezet felvázolja a rendszer elvárt specifikus funkcionális képességeit, azokat a nem-funkcionális minőségi attribútumokat, amelyeknek meg kell felelnie, valamint a felhasználói interakciót meghatározó elsődleges használati eseteket.

3.1 Funkcionális követelmények

A funkcionális követelmények határozzák meg azokat a pontos viselkedéseket, folyamatokat és funkciókat, amelyeket a Smart Turntable Assistant-nek végre kell hajtania, hogy értéket teremtsen a végfelhasználó számára.

- **Valós idejű audio azonosítás:** A rendszernek folyamatosan figyelnie kell a bejövő audio streamet, és automatikusan detektálnia kell a lejátszás kezdetét. Az észlelés után rögzítenie kell egy rövid audio buffert, és külső audio fingerprinting API-kat kell lekérdeznie a sáv azonosításához.
- **Metaadat-bővítés:** Sikeres audio egyeztetés után az alkalmazásnak másodlagos zenei adatbázisokat kell lekérdeznie. Ez a lépés szükséges a nagy felbontású albumborítók, a pontos sáv hosszúság (milliszekundumban), és a szabványosított albumnév lekéréséhez.
- **Szinkronizált dalszöveg megjelenítés:** A backendnek le kell töltenie az időbélyeggel ellátott dalszövegfájlokat az azonosított zeneszámhoz. A frontendnek értelmeznie kell ezeket az időbélyegeket, és automatikusan

kell görgetnie a szöveget, szinkronban az élő lejátszási időzítővel. A felhasználónak lehetőséget kell biztosítani a szinkronizációs idővonal manuális eltolására az apró mechanikai késleltetések korrigálása érdekében.

- **Hardverállapot-diagnosztika:** A szoftvernek elemeznie kell a bejövő analóg jelet az effektív (RMS) hangerőszint kiszámításához. Matematikailag fel kell dolgoznia a hangot a hirtelen felületi kattogások detektálására, az alacsony frekvenciás mechanikai motorrezgés mérésére, és a magas frekvenciás sziszegő torzítás (sibilance) számszerűsítésére.
- **Hangszedő életciklus kezelés:** A rendszernek fenn kell tartania egy adatbázist a felhasználó tulajdonában lévő lemezjátszó tükrről. Nyomon kell követnie az éppen aktív hardver kumulatív lejátszási idejét, lehetővé kell tennie a felhasználó számára a maximálisan ajánlott élettartam meghatározását, és biztosítani kell a számláló nullázásának lehetőségét a hardver cseréjekor.
- **Digitális hallgatási naplók:** Az alkalmazásnak támogatnia kell a hitelesítést külső, közösségi zene-nyomonkövető szolgáltatásokkal, mint amilyen a Last.fm. Automatikusan naplózni kell a zeneszámot a felhasználó profiljába, amint a lejátszás eléri a meghatározott időtartam-küszöböt.
- **Esztétikai testreszabás:** A webes felületnek lehetővé kell tennie a felhasználók számára, hogy testreszabják a digitális lemez vizuális megjelenését a színes fizikai hordozóknak megfelelően. Ezt a színpreferenciát az adatbázisba kell menteni, és automatikusan újra kell alkalmazni, valahányszor az adott albumról szóló zeneszám lejátszásra kerül.

3.2 Nem-funkcionális követelmények

A nem-funkcionális követelmények diktálják azokat a teljesítményszabványokat, megbízhatósági korlátokat és használhatósági metrikákat, amelyeket a rendszerarchitektúrának ki kell elégítenie.

- **Teljesítmény és késleltetés:** A háttérben futó audiofeldolgozó szálnak folyamatosan működnie kell anélkül, hogy blokkolná a webszerver műveleteit. A diagnosztikai adatokat másodpercenként többször kell a frontend felé sugározni a folyékony vizuális animációk és a pontos VU Meter-ek, hangszint jelzők.

- **Megbízhatóság és Fallback mechanizmusok:** A külső adatbázisok felé irányuló hálózati lekérdezéseknek robusztus fallback stratégiákat kell alkalmazniuk. Ha egy elsődleges felismerő motor vagy metaadat-szolgáltatás meghibásodik, a rendszernek zökkenőmentesen meg kell próbálnia a másodlagos források használatát az adatvesztés elkerülése és a folyamatos működés biztosítása érdekében.
- **Használhatóság és reszponzivitás:** A grafikus felhasználói felületnek dinamikusan kell skálázódnia a különböző kijelzőméretek között, a mobiltelefonok képernyőitől a nagy felbontású asztali monitorokig. Konténer-lekérdezéseket és folyékony tipográfiát kell alkalmazni az elrendezés integritásának megőrzéséhez.
- **Adatbázis-konkurencia:** A mögöttes relációs adatbázisnak támogatnia kell az egyidejű olvasási és írásai műveleteket. A háttérszálaknak képesnek kell lenniük a hangszedő statisztikáinak frissítésére, miközben az azonosító szál egyidejűleg új hallgatási előzményeket ment, amely az adatbázis-zárolási hibák megelőzése érdekében Write-Ahead Logging módot igényel.

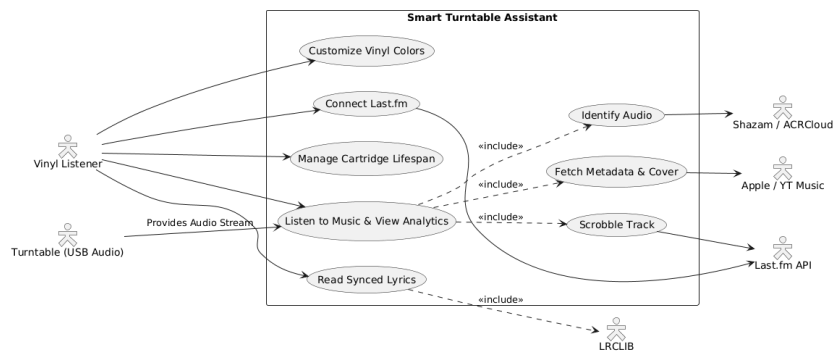
3.3 Használati esetek

A következő használati esetek a hallgató és a Smart Turntable Assistant közötti elsődleges interakciókat szemléltetik egy tipikus zenehallgatási munkamenet során.

- **Automatizált lejátszás-monitorozás:** A felhasználó ráengedi a tűt egy bakelitlemezre. A rendszer detektálja az audio küszöbértéket, azonosítja a számot, és automatikusan, manuális beavatkozás nélkül feltölti a dashboardot a megfelelő albumborítóval és a szinkronizált dalszöveggel.
- **Hardverdiagnosztika megfigyelése:** A felhasználó a diagnosztika fülre navigál, miközben egy lemez szól. Figyeli az élő felületet, hogy ellenőrizze a lemezjátszó motorjának megfelelő szigetelését, és figyeli a click counter-t annak megállapítására, hogy a bakelit felülete tisztítást igényel-e.
- **Hangszedő konfigurációja:** A felhasználó megnyitja személyes profilját egy újonnan vásárolt tű regisztrálásához. A gyártó ajánlásai alapján beállítja a várható élettartamot, aktív hardverként jelöli meg,

majd visszatér a dashboardra, hogy figyelemmel kísérje a hátralévő élettartam százalékos arányát.

- **Vizuális felület testreszabása:** Miközben a felhasználó egy album áttetsző piros bakelit kiadását hallgatja, a digitális vizualizálóval interakcióba lépve megváltoztatja a megjelenítési stílust, hogy az illeszkedjen a fizikai hordozóhoz. A rendszer ezt a preferenciát a memóriába menti, és automatikusan alkalmazza ugyanezt a vizuális stílust, amikor a jövőben ebből az albumból származó szám kerül lejátszásra.



3.1. ábra: UML Use Case Diagram illustrating user and system interactions.

4. fejezet

Rendszerterv és architektúra

Egy folyamatos analóg audiojel strukturált, interaktív digitális élménnyé alakítása erősen szétválasztott és konkurens rendszerarchitektúrát igényel. Ez a fejezet a Smart Turntable Assistant szerkezeti felépítését részletezi, felvázolva a kliens-szerver kommunikációs protokollokat, a relációs adatbázis sémáját, valamint azt az összetett szálkezelést, amely a valós idejű telemetria és az aszinkron hálózati kérések egyidejű feldolgozásához szükséges.

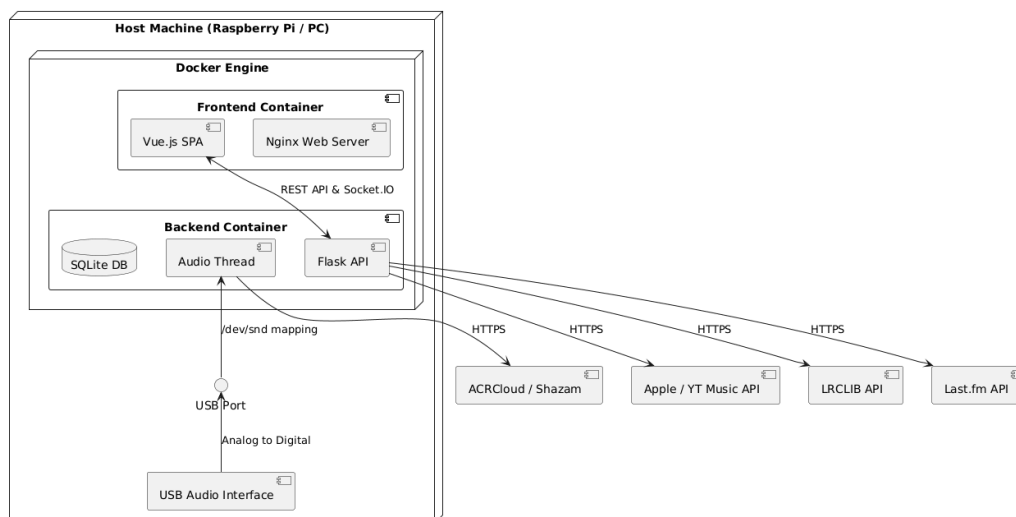
4.1 Kliens-Szerver architektúra

Az alkalmazás architektúrája egy robusztus kliens-szerver modellt követ, amelyet mind a perzisztens adatműveletek, mind a nagyfrekvenciás valós idejű telemetria kezelésére terveztek. A backendet a Python és a Flask mikrokeretrendszer hajtja meg, amely központi feldolgozó egységként működik, és az Advanced Linux Sound Architecture (ALSA) valamint a PulseAudio alrendszeren keresztül közvetlenül kapcsolódik a host audio hardveréhez.

A frontend egy Vue.js keretrendszerrel épített Single Page Application (SPA), amely interaktív dashboardként szolgál a felhasználó számára. A kommunikáció e két eltérő környezet között kétirányú. A szabványos állapotváltozásokat – mint például a felhasználói hitelesítés, a hardveres konfigurációk frissítése és a manuális szinkronizációs beállítások – állapotmentes REST (Representational State Transfer) API végpontok kezelik.

Ezzel szemben az audiodiagnosztika és a lejátszási időzítők folyamatos áramlása állandó, alacsony késleltetésű kapcsolatot tesz szükségessé. Ezt WebSockets technológiával, a Socket.IO könyvtár segítségével értük el. Egy full-duplex kommunikációs csatorna kiépítésével a backend másodpercenként többször képes a hangerőre, a rumble-re és a sibilance-re vonatkozó matematikai számításokat a frontend felé tolni, biztosítva, hogy a felhasználói felület

rendkívül reszponzív maradjon, és tökéletes szinkronban legyen a fizikai lemezjátszóval.



4.1. ábra: Deployment diagram amely a docker image felépítését mutatja be.

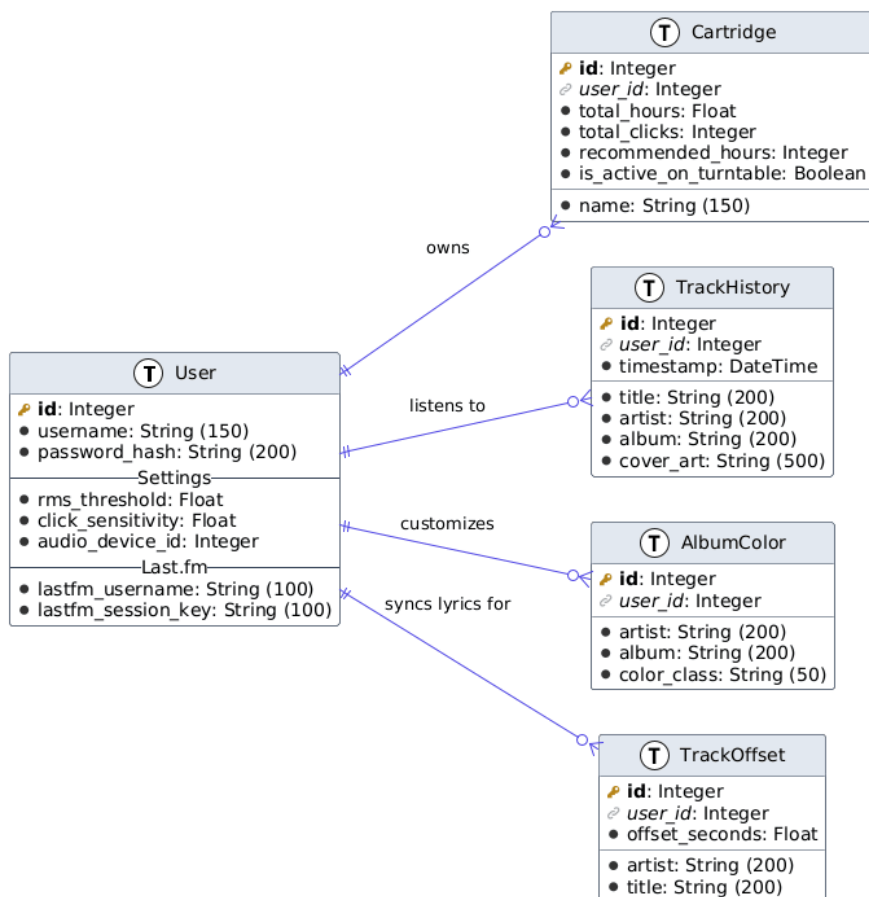
4.2 Adatbázis-tervezés

A perzisztens adattárolást egy relációs adatbáziskezelő rendszer, konkrétan az SQLite kezeli, a SQLAlchemy ORM (Object-Relational Mapper) integrálásával. A séma normalizált, így a felhasználói hitelesítő adatok, a hardveres metrikák és a historikus zenehallgatási szokások biztonságosan elkülönülnek.

A séma magjában a **User** (felhasználó) entitás áll, amely egy-a-többhöz kapcsolatot tart fenn az összes többi táblával, biztosítva az adatok izolációját egy többfelhasználós környezetben. A **Cartridge** (Hangszedő) tábla a felhasználó tulajdonában lévő fizikai lemezjátszó tűket követi nyomon, tárolva azok maximálisan ajánlott élettartamát, valamint a lemezjátszón aktívan eltöltött, kumulatív üzemórákat.

A **TrackHistory** tábla digitális hallgatási naplóként funkcionál, amely minden egyes sikeres audio felismerési eseménynél rögzíti az azonosított előadót, az albumot és a nagy felbontású borítókép URL-jét (Uniform Resource Locator). Az alkalmazás esztétikai és szinkronizációs funkcióinak támogatása érdekében az **AlbumColor** és a **TrackOffset** táblák a felhasználó által meghatározott vizuális preferenciákat és a dalszöveg-időzítési kalibrációkat tárolják, kifejezetten az azonosított előadóhoz és albumhoz kapcsolva azokat. Ez a relációs struktúra biztosítja, hogy amikor a felhasználó bejelentkezik, az aktív tű

limiteit és a korábbi zeneszám-módosításait a rendszer azonnal lekérje, és alkalmazza a globális alkalmazás-állapotr.



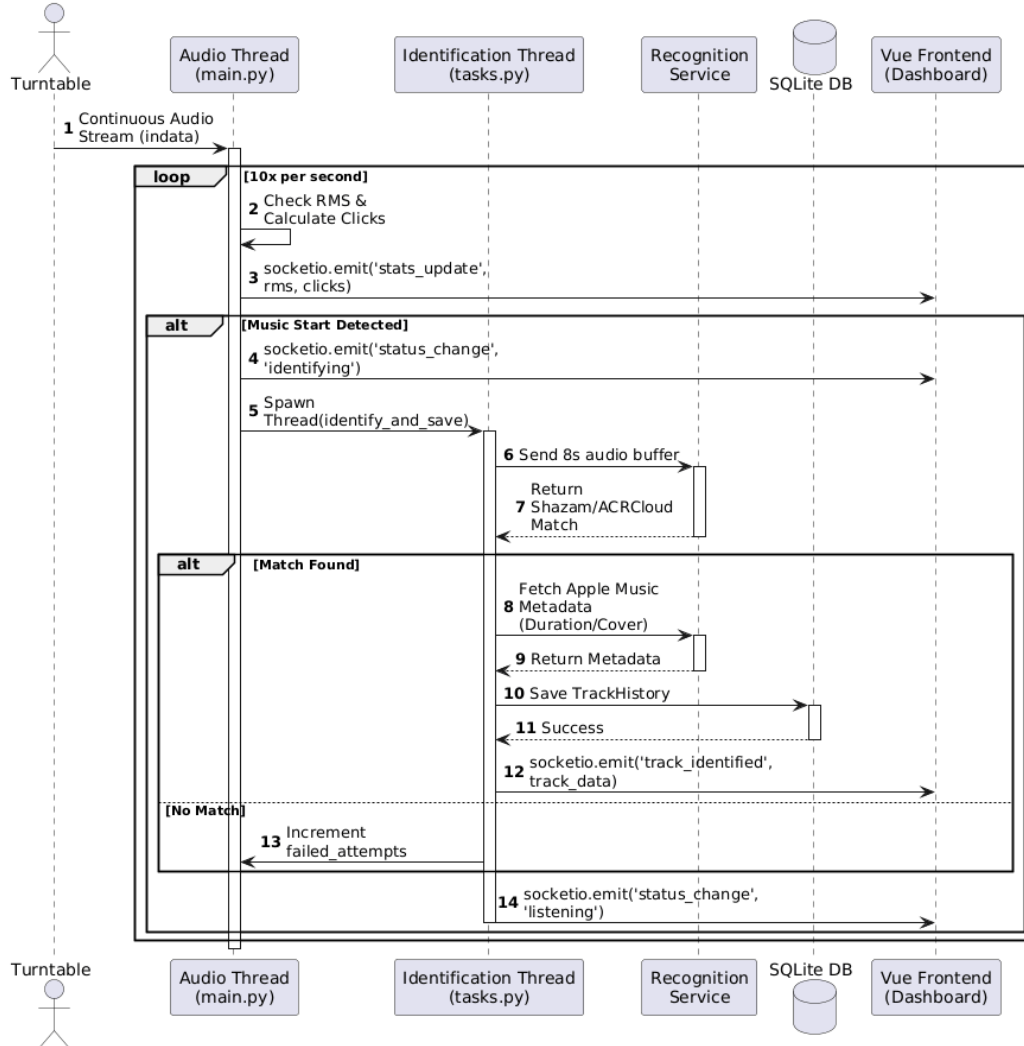
4.2. ábra: Entity-Relationship Diagram az adatbázis sémáról.

4.3 Konkurencia és szálkezelés

Az élő audio streamek elemzése és a külső webszolgáltatások egyidejű lekérdezése jelentős konkurencia-kihívást jelent. Egy tisztán szinkron megközelítés azt eredményezné, hogy az alkalmazás blokkolná az összes diagnosztikai számítást, és lefagyasztaná a felhasználói felületet, amíg egy audio fingerprinting szolgáltatás hálózati válaszára vár. Ennek megoldására a backend egy többszálú architektúrát alkalmaz, amelyet egy centralizált globális állapot-objektum köré szerveztek.

Az elsődleges audiofeldolgozó szál folyamatosan fut a háttérben. Különálló blokkokban olvassa be a hangot a hardveres pufferból, kiszámítja az RMS (Root Mean Square) értékeket, detektálja a felületi kattogásokat (clicks), és frissíti az aktív hangszedő használati statisztikáit. Amikor ez az elsődleges szál a kitartott hangerő-küszöbértékek alapján matematikailag megállapítja, hogy egy új dal kezdődött, elindít egy független, párhuzamos azonosító szálát.

Ez a másodlagos szál rögzít egy ideiglenes hangmintát, és kezeli a fingerprinting motorok, valamint a metaadat-bővítő API-k felé irányuló, késleltetett hálózati kéréseket. Mivel több szál lép egyszerre interakcióba az adatbázissal, az SQLite adatbázis Write-Ahead Logging (WAL) módra van konfigurálva. Ez a fejlett konkurencia-mód lehetővé teszi, hogy az elsődleges szál biztonságosan mentse a hangszedő kopási statisztikáit a lemezre, miközben a másodlagos azonosító szál aktívan írja az új hallgatási előzmény-rekordokat, ezáltal teljesen kiküszöbölve az adatbázis-zárolási hibákat, és biztosítva a zökkenőmentes háttérműködést.



4.3. ábra: UML Sequence Diagram a többszálú kezelésről.

5. fejezet

Megvalósítási részletek

Az architekturális terv funkcionális alkalmazássá alakítása precíz programozási módszertanokat, robusztus hibakezelést és hatékony algoritmustervezést igényel. Ez a fejezet a Smart Turntable Assistant konkrét implementációját részletezi, feltárva a digitális jelfeldolgozás (DSP) matematikáját, a lépcsőzetes metaadat-lekérdező logikát és a reaktív frontend renderelési technikákat.

5.1 Digitális jelfeldolgozás (Digital Signal Processing)

Az alkalmazás magja a lemezjátszó által biztosított analóg audiojel folyamatos elemzésén alapul. A Python backend a `sounddevice` és a `NumPy` könyvtárak felhasználásával, diszkrét matematikai tömbökben rögzíti a hangot. Ezeket az adatblokkokat a rendszer másodpercenként többször feldolgozza a jelentéssel bíró diagnosztikai metrikák kinyerése érdekében.

Annak detektálására, hogy a felhasználó mikor teszi rá a tűt a lemezre, a rendszer kiszámítja az audio chunk RMS értékét, amely az átlagos akusztikus energiát reprezentálja. Ha ez az energia egy megadott ideig a felhasználó által definiált küszöbérték felett marad, az alkalmazás regisztrál egy "lejátszás kezdete" (playback start) eseményt.

A felületi tökéletlenségek, mint a por és a karcok, hirtelen, magas frekvenciájú tranziensekként jelennek meg, amelyeket kattogásnak vagy sercegésnek (clicks or pops) nevezünk. A szoftver ezeket úgy detektálja, hogy kiszámítja az audiojel első deriváltját, amely felüláteresztő szűrőként (high-pass filter) funkcionál a gyors változások izolálására. Ezen változások szórásának kiszámításával az algoritmus azonosítja azokat a statisztikai kiugró értékeket, amelyek meghaladnak egy adott érzékenységi szorzót, sikeresen megszámlálva ezáltal a diszkrét felületi zajokat.

A fejlettebb diagnosztikák közé tartozik a mechanikai motorzaj (rumble) és a sziszegés (sibilance) nyomon követése. A rumble kiszámításához az alkalmazás Fast Fourier Transzformációt (FFT) alkalmaz az audio tömbön, elszigetelve és összegezve a kizárólag tíz Hertz és ötven Hertz közötti akusztikus energiát. A sibilance-t, amely a magas frekvenciájú követési torzítást (tracking distortion) jelzi, úgy méri, hogy a jelet átvezeti egy Butterworth sávszűrőn (bandpass filter), amely az öt kilohertztől tíz kilohertzig terjedő tartományt célozza meg. Ennek az izolált sávnak az energiáját arányosítja a teljes jelenlegi energiával, ami egy olyan százalékos értéket ad eredményül, amely dinamikusan vezérli a frontend felületén lévő "harshness", torzítás mérőt.

5.2 A vízesés elvű metaadat-stratégia

Ha a zenefelismerést és a metaadatokat kizárólag egyetlen külső szolgáltatásra bízunk, az gyakran hiányos információkat eredményez, különösen az underground vagy rétegzenei műfajok esetében. Ennek megoldására az alkalmazás egy cascading waterfall stratégiát alkalmaz, amely szigorú prioritási sorrendben kérdez le több API-t.

A folyamat egy nyolcmásodperces hangminta rögzítésével kezdődik, amelyet a rendszer aszinkron módon elküld az audio fingerprinting motoroknak. Ha találatot regisztrál, a rendszer kinyeri a payload-ba (válaszadathalmazba) ágyazott egyedi Apple Music azonosítót. Ez a specifikus azonosító lehetővé teszi a backend számára, hogy közvetlenül lekérdezze az iTunes web-szolgáltatást, garantálva a pontos sáv hossz (milliszekundumban), a hivatalos albumnév és egy nagy felbontású, négyzet alakú albumborító sikeres letöltését.

Ha az elsődleges fingerprinting szolgáltatás nem biztosít pontos azonosítót, a rendszer visszalép egy szöveg alapú keresésre a `ytmusicapi` könyvtár segítségével. Ez a könyvtár a hivatalos YouTube Music katalógust olvasza le. Annak érdekében, hogy a frontend nagy felbontású grafikát kapjon az alacsony felbontású videó indexképek helyett, a szoftver manipulálja a dinamikusan generált kép URL-jét: lecseréli az alapértelmezett dimenzió-paramétereket szigorú, nagy felbontású kérésekre, így tökéletesen éles képeket biztosítva a felhasználói felület számára.

5.3 Backend állapotkezelés és adat-streamelés

Az adatáramlás kezelése a párhuzamos háttérzálak és a csatlakoztatott webes kliensek között egy központosított igazságforrást, úgynevezett source of truth-t igényel. A backend egy globális állapot objektumot valósít meg, amely

tartalmazza az aktuális zeneszám metaadatait, az élő diagnosztikai értékeket és a lejátszási időzítőket.

Mivel az audiofeldolgozó szál másodpercenként tízszer értékeli ki az adatokat, a hangszedő-használati statisztikák közvetlenül az SQLite adatbázisba történő írása ilyen gyakorisággal súlyos I/O szűk keresztmetszeteket okozna, és rontaná a tárolóeszköz élettartamát. Ehelyett a rendszer az eltelt lejátszási másodperceket és a kattogások számát ideiglenes memóriaváltozókban halmozza fel. Egy szabályozó mechanizmus gondoskodik arról, hogy ezek a felhalmozott értékek csak tíz másodpercenként kerüljenek elmentésre a fizikai adatbázisba. Ezzel egyidejűleg az élő értékeket a Socket.IO kapcsolaton keresztül a rendszer kisugározza az összes csatlakoztatott kliens felé, biztosítva, hogy a frontend folyékony, valós idejű animációkat jelenítsen meg anélkül, hogy túlterhelné a szerver hardverét.

5.4 Frontend interfész és reaktív architektúra

A felhasználói felületet egy Single Page Application-ként (SPA) terveztük meg a Vue.js keretrendszer használatával, azzal a céllal, hogy függetlenítse a vizuális megjelenítést a backend logikától.

5.4.1 Kliens állapot és Útválasztás

A globális kliens állapotot – beleértve a felhasználói hitelesítést és a session tokeneket – a Pinia állapotkezelő könyvtár (state management library) kezeli. Ez biztosítja, hogy amikor a felhasználó frissíti az oldalt, a munkamenete megmarad, mivel a rendszer a hitelesítő adatokat közvetlenül a böngésző helyi tárhelyéből (`localStorage`) olvassa ki. A Vue Router elfogja az összes navigációs kísérletet, és ellenőrzi a hitelesítési állapotot, mielőtt hozzáférést biztosítana az elsődleges dashboardhoz vagy a felhasználói profil konfigurációs oldalaihoz.

5.4.2 Dinamikus stílusok és Témák

A grafikus felület teljes mértékben reszponzív módon épült fel, modern CSS (Cascading Style Sheets) funkciókat kihasználva, mint például a konténerlekérdezéseket és a folyékony tipográfiai `clamp` függvényeket. Ez garantálja, hogy a vizuális lemezjátszó és a szöveges elemek arányosan méreteződjenek, függetlenül attól, hogy mobilkészüléken vagy nagy felbontású asztali monitoron tekintik meg azokat.

Az esztétikai megjelenést a gyökér dokumentum elemre alkalmazott dinamikus adatattribútumok szabályozzák. Ezen attribútumok átváltásával az alkalmazás azonnal lecseréli a CSS változók kiterjedt készletét. Ez a mechanizmus lehetővé teszi a felhasználó számára, hogy a felületet egy modern, lekerekített, finom színátmenetekkel rendelkező esztétikáról egy dobozos, erősen kontrasztos retro felületre váltsa, teljesen átalakítva a felhasználói élményt az oldal újratöltése nélkül.

5.4.3 Szinkronizált dalszöveg-renderelés

Amikor a rendszer azonosít egy zeneszámot, a backend lekérdez egy nyílt forráskódú dalszöveg-adatbázist, hogy lekérje a pontos szinkronizációs időbélyegekké formázott szöveget. A frontend ezt a szövegkarakterláncot egy matematikai objektumokból álló tömbbé értelmezi. Egy reaktív számított tulajdonság folyamatosan összehasonlítja ezeket az időbélyegeket a backend által sugárzott élő lejátszási időzítővel.

Amikor az időzítő meghalad egy dalszöveg-időbélyeget, az aktív index frissül, és az alkalmazás egy finom görgetési eseményt vált ki. A szoftver a Vue keretrendszer `nextTick` renderelési ciklusát használja annak biztosítására, hogy a Document Object Model (DOM) teljes mértékben frissüljön, mielőtt megkísérelné az aktív sort közvetlenül a nézetablak közepére görgetni.

5.5 Harmadik féltől származó integrációk

Az analóg lejátszás és a digitális nyomon követés közötti szakadék áthidalása érdekében a szoftver erősen integrálódik a Last.fm közösségi zenei platformmal. Az alkalmazás egy webes hitelesítési folyamatot valósít meg, ahol a felhasználó engedélyezi a szoftvert, amely egy ideiglenes token-t ad vissza.

Mivel a szabványos Python könyvtárak a szigorú formázási szabályok miatt gyakran nem tudják helyesen értelmezni ezt a token-t, a backend a hitelesítési lépéshez megkerüli a harmadik féltől származó burkolókat. Ehelyett manuálisan állít össze egy MD5 algoritmussal hashelt biztonsági aláírást, amely tartalmazza a fejlesztői `secret`-eket és a token-t, majd közvetlen hálózati kérést küld a Last.fm szervereinek egy végleges `session key` lekérésére. Lejátszás közben egy háttérzál figyeli az eltelt sávidőt. Amikor a zeneszám eléri a pontosan ötven százalékos befejezettséget, vagy a négy perc folyamatos lejátszást, egy másodlagos háttérfolyamat biztonságosan továbbítja az előadó és a cím adatait a Last.fm szervereinek, sikeresen katalogizálva ezzel az analóg hallgatási munkamenetet a felhasználó digitális profiljában.

6. fejezet

Tesztelés és Értékelés

Annak biztosítása érdekében, hogy a Smart Turntable Assistant megbízhatóan működjön valós körülmények között, egy szigorú tesztelési és optimalizálási fázist hajtottunk végre. Ez a fejezet a hardver- és szoftverkomponensek integrációs tesztelését részletezi, valamint azokat a specifikus teljesítmény-optimalizációkat, amelyek a folyamatos audio-elemzés futtatásához szükségesek a korlátozott erőforrású számítástechnikai eszközökön.

6.1 Manuális és integrációs tesztelés

A rendszert különféle fizikai bakelitlemezekkel értékeltük, amelyek különböző műfajokat, préselési minőségeket és felületi állapotokat fedtek le. Az audio bemeneti streamet szigorúan vizsgáltuk a digitális jelfeldolgozó algoritmusok pontosságának ellenőrzése céljából. Ismert fizikai sérülésekkel rendelkező lemezeket játszottunk le annak megerősítésére, hogy a click detection logika pontosan növeli a hiba-számlálót anélkül, hogy az agresszív dob tranzienseknél false positive találatokat regisztrálna. Hasonlóképpen, a sibilance és a rumble mérőket olyan lemezekkel ellenőriztük, ahol ismert volt a követési torzítás, illetve szándékos motorrezgéssel rendelkező lemezjátszó-beállításokkal is teszteltünk, igazolva a frekvenciaelemzés matematikai érvényességét.

A frontend interfész és a backend szerver közötti integrációt a grafikus felhasználói felület aktív lejátszás közbeni manipulálásával teszteltük. A manuális sávdetektálás elindítása, a szinkronizált dalszöveg-idővonal törtmásodpercekkel történő beállítása, valamint a vizuális témák gyors váltása is hibátlanul teljesített anélkül, hogy megszakította volna a folyamatos WebSocket telemetria-sugárzást. A lépcsőzetes metaadat-lekérdező rendszer sikeresen azonosította az obskúrus számokat azáltal, hogy másodlagos zenei

adatbázisokra támaszkodott, amikor az elsődleges audio fingerprinting motor nem szolgáltatott teljes információt.

6.2 Teljesítmény-optimalizáció

A folyamatos audio-elemző algoritmusok és a webszerver egyidejű futtatása korlátozott hardveren szigorú erőforrás-kezelést igényel. A kezdeti tesztelés feltárta, hogy a hangszedő egészségügyi statisztikáinak másodpercenként többszöri adatbázisba írása súlyos I/O szűk keresztmetszeteket okozott, ami végül az alkalmazás összeomlásához vezetett az adatbázis zárolási hibái miatt.

Ennek megoldására az adatbázis-motort úgy konfiguráltuk át, hogy Write-Ahead Logging (WAL) módot használjon, amely egy fejlett naplózási mód, ami lehetővé teszi az egyidejű olvasási és írási műveleteket a különböző háttérzálakon. Ezenkívül egy memóriapufferelő rendszert is implementáltunk a fő végrehajtási hurokban. Ahelyett, hogy folyamatosan interakcióba lépne az adatbázissal, az elsődleges audio szál az eltelt lejátszási másodperceket és a kattogási eseményeket ideiglenes változókból halmozza fel. A rendszer a kombinált végösszegeket csak tíz másodpercenként írja ki a perzisztens tárolómeghajtóra. Ez az optimalizálás jelentősen csökkentette a processzor terhelését és minimalizálta a fizikai írási ciklusokat, biztosítva a szilárdtest-meghajtó (SSD) hosszú élettartamát, miközben fenntartotta a rendkívül reszponzív felhasználói élményt.

7. fejezet

Összegzés és jövőbeli fejlesztések

Az utolsó fejezet összefoglalja a fejlesztési folyamat eredményeit, értékelve a megvalósított funkciók sikerét a kezdeti célkitűzésekhez képest, és feltárja a jövőbeli kutatás és a szoftverbővítés lehetséges útjait.

7.1 Az elért eredmények értékelése

A Smart Turntable Assistant sikeresen eléri elsődleges célját, az analóg bakelit-lemez-hallgatási élmény modernizálását. A nyers elektromos audiojel passzív monitorozásával a rendszer áthidalja a fizikai média és a digitális ökoszisztéma közötti szakadékot. Az alkalmazás közel 80% pontossággal azonosítja a lejátszott sávokat, a hallgatási munkamenetet nagy felbontású borítóképekkel és szinkronizált dalszövegekkel gazdagítja, és zökkenőmentesen naplózza a hallgatási előzményeket a külső közösségi zenei platformokra.

Továbbá, az audiofil-szintű diagnosztikai eszközök beépítése, valós idejű vizuális visszajelzést biztosít a felhasználóknak a tű és a lemezek fizikai állapotáról. A rendkívül moduláris, szétválasztott (decoupled) architektúra biztosítja, hogy a mögöttes Python backend és a reaktív webes interfész hatékonyan működjön, egy olyan professzionális és robusztus szoftveres megoldásban, amely tiszteletben tartja a hagyományos hanglemez élményt, miközben modern digitális kényelmet nyújt.

7.2 Jövőbeli fejlesztési lehetőségek

Bár a jelenlegi implementáció teljesíti az összes kezdeti követelményt, számos lehetőség kínálkozik a jövőbeli továbbfejlesztésre. A legjelentősebb fejlődés

saját neuron hálók használatával való silince detection és két zeneszám közti különbség (tempó és hangszín) detektálásával lenne elérhető.

Ezenfelül a diagnosztikai csomag kibővíthető lenne egy automatizált lemezjátszó-kalibrációs móddal is. Egy lemez lejátszásával ki tudja választani a felhasználó, hogy hol megy már a zene, ezzel a music detection algoritmus érzékenységét egyből be tudja állítani egy elfogadható állapotra.